

Stackademic · [Follow publication](#)

★ Member-only story

15 Git Terms That Confuse Developers (and What They Actually Mean)

6 min read · Oct 22, 2025



Subodh Shetty

Follow



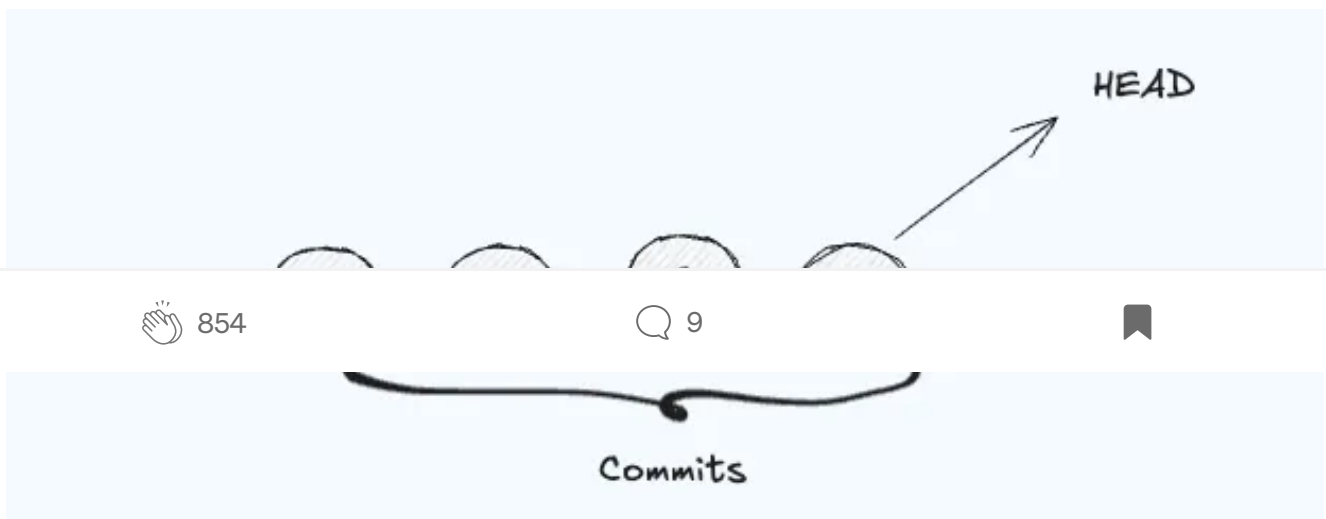
Listen



Share



More



854



9



What is HEAD?

I've been in the trenches for 17 years now - reviewed hundreds of pull

requests, fixed a fair share of messy repos, and mentored devs who got tangled up in Git commands they didn't fully understand. And honestly, I don't blame them. Git is powerful, but it's also full of jargon that looks similar but behaves differently.

So let's cut the noise and go through the Git terms that even experienced devs sometimes misuse.

If you're a junior dev or anyone starting fresh, this list will save you hours of head-scratching later.

HEAD **vs** head **vs Detached** HEAD

What devs get wrong:

People think HEAD is just another branch name.

What it really is:

HEAD is a *pointer* to your current commit - usually the tip of the branch you're on.

When you commit, HEAD moves to that new commit.

```
# Shows what HEAD points to
git log --oneline --decorate -n 3

# --oneline Condenses each commit into a single line:
# The first 7 characters of the commit hash &
# The commit message

# --decorate
# Adds branch and tag references beside commits
# so you can see where pointers like HEAD, main, origin/main, or v1.0.0

# -n 3
# Limits output to the last 3 commits.
```

You'll see something like:

```
a3c4d5e (HEAD -> main, origin/main) Add user API
```

If you check out an old commit directly:

```
git checkout a3c4d5e
```

you'll see the message `You are in 'detached HEAD' state.`

That just means you're not on a branch - you're looking at a historical snapshot. Nothing's broken.

If you want to save work from there:

```
git switch -c hotfix/legacy-bug
```

head (lowercase), on the other hand, isn't a Git keyword - it's often used informally or as a plural, like "the heads of branches."

Git stores branch tips under `.git/refs/heads/`, which is why you might see that word show up in internal references.

For example:

```
.git/refs/heads/main  
.git/refs/heads/feature/login
```

So when you see “all heads,” it means “the latest commit of each branch,” not the special `HEAD` pointer.

Now the suffixes:

Git gives you ways to *walk back through history* starting from `HEAD`.

`HEAD~1`

Means “one commit before `HEAD`.”

`HEAD~2` means “two commits before,” and so on.

```
git show HEAD~1
```

Shows the commit just before your current one.

`HEAD^1` **and** `HEAD^2`

These are *parent pointers*. They matter most for merge commits.

A merge commit has two parents - one from the branch you were on, and one from the branch you merged in.

```
git diff HEAD^1 HEAD^2
```

Compares what each side contributed to the merge.

`^1` means “first parent,” `^2` means “second parent.”

Rule of thumb:

- Use `~` to walk backward linearly.
- Use `^` when dealing with parents of a merge commit.

Example walkthrough:

After merging feature into main, the merge commit (M) has two parents:

- `HEAD^1` → commit `C` (main before merge)
- `HEAD^2` → commit `E` (end of feature branch)

Once you visualize *HEAD* as you are here, *Git's navigation model* clicks instantly.

2. Commit vs Changeset

A lot of devs think a commit *is* the diff - it's not.

- A **commit** = full snapshot of the repo + metadata + parent links

- A **changeset** = difference *between* two commits.

```
git show HEAD          # commit with its diff (changeset view)
git diff HEAD~1 HEAD   # only the raw diff between two commits
git add -p             # stage changes selectively (build your changeset)
git commit -m "Fix NPE in user lookup"
```

Understanding this difference helps you craft atomic commits - one logical change per commit - instead of dumping everything at once.

3. Branch ≠ Copy

Creating a branch doesn't clone the codebase.
It just creates a new pointer to a commit.

```
git branch feature/login
git switch feature/login
```

Now you have a lightweight pointer diverging from `main`. No extra files, no duplication.

4. Tags

Tags mark specific commits - usually releases and don't move.

```
git tag v1.0.0
git push origin v1.0.0
```

Unlike branches, tags are *immutable*. Once set, they always point to the same commit.

5. Fast-Forward Merge

“Fast-forward” isn’t a special operation - it’s just a merge with no divergence.

No divergence -> Fast-forward possible

`main` hasn’t moved since `B`.

```
git switch main  
git merge --ff-only feature
```

Result:

No merge commit created -> Git simply moved `main`'s pointer forward.

Diverged branches -> Fast-forward not possible

Both advanced after `B`.

```
git switch main  
git merge feature
```

Now Git must create a merge commit to combine `E` and `D`.

If you insist on fast-forward:


```
git merge --ff-only feature
# error: Not possible to fast-forward
```

Rule:

If both sides have commits after they split, it's not fast-forwardable.

7. Squash

Squash flattens multiple commits into one before merging -> good for cleaning up noisy feature branches.

```
git merge --squash feature/login
git commit -m "Add login feature"
```

Your branch history stays clean without losing the work context in PRs.

8. origin vs upstream

This is an interesting one.. I came to know the difference just a few days back when I was working on a forked project. All this while, i was presuming both to be same :)

`origin` is the default name of your remote when you clone a repo.

If you fork a repo, your fork's main repo is called `upstream`.

```
git remote -v
```

You might see:

```
origin  git@github.com:yourname/project.git
upstream git@github.com:org/project.git
```

Use `upstream` to pull in changes from the original source:

```
git fetch upstream
```

9. Remote-Tracking Branches

`origin/main` is not the same as `main`.

- `main` = your local branch
- `origin/main` = Git's last known snapshot of `main` from the remote

```
git fetch
git diff main origin/main
```

This tells you what's changed upstream without merging yet.

10. Fetch vs Pull

`git fetch` only updates your local knowledge of remotes.

`git pull` does that *and* merges (or rebases) automatically.

```
git fetch origin  
git merge origin/main
```

That's safer and more explicit than a blind `git pull`.

11. The Staging Area (Index)

The staging area is Git's "waiting room."

You decide which changes go into the next commit.

```
git add src/App.java  
git status  
git commit -m "Fix null check"
```

You can stage specific hunks:

```
git add -p
```

It's like filling a shopping cart before checkout.

12. Working Directory

Your working directory is just the files you're editing -> not the repo itself.

The actual repository lives inside `.git/`.

```
git status
```

```
# "working tree clean" means your files match HEAD
```

13. Tracked vs Untracked Files

Files you've added once are "tracked."

New files aren't until you stage them.

```
git status  
# Untracked files: (use "git add <file>")
```

Always check this before committing -> untracked files won't appear in commits.

14. Dirty Tree

"Dirty tree" just means your working directory has uncommitted changes.

```
git status  
# modified: src/App.java
```

Commit or stash them before switching branches to avoid conflicts.

15. Reset vs Revert

The two most misunderstood commands in Git.

- `reset` moves HEAD and can discard commits (dangerous).
- `revert` creates a new commit that undoes previous changes (safe).

```
git reset --hard HEAD~1    # deletes last commit
git revert HEAD             # undoes safely by adding new commit
```

If you're working on a shared branch, always use `revert`.

Final Thoughts

At its core, Git is just a graph of commits and pointers.

Once you understand how `HEAD`, branches, and snapshots work, everything else feels logical.

A message from our Founder

Hey, Sunil here. I wanted to take a moment to thank you for reading until the end and for being a part of this community.

Did you know that our team run these publications as a volunteer effort to over 3.5m monthly readers? **We don't receive any funding, we do this to support the community.** ❤️

If you want to show some love, please take a moment to **follow me on LinkedIn, TikTok, Instagram**. You can also subscribe to our **weekly newsletter**.

And before you go, don't forget to **clap** and **follow** the writer!

Technology

Programming

Software Development

Git

Software Engineering



Follow

Published in Stackademic

80K followers · Last published 1 day ago

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.



Follow

Written by Subodh Shetty

723 followers · 66 following

Engineering Manager | AWS Solution Architect | Writing on tech, life, and the patterns that connect it all | subodhshetty.com

Responses (9)



To respond to this story,
get the free Medium app.

Continue in app



Arun Das

Oct 23, 2025



If you're scared of Git, you're not a developer. I knew all 15 of these, exactly what they mean, and I use Git mostly from command line. Also why paywall this basic info? I could just write the same and publish it for free, and you can't claim... [more](#)



66



3 replies



Shaant he/him

Oct 22, 2025



HEAD is a pointer to your current commit - usually the tip of the branch you're on.

Subodh, knowing HEAD is key. Many forget it moves with every commit. Makes Git less scary.



10



Andrew Selby

Oct 27, 2025



You might want to check that first AI generated image - it identifies HEAD as being the leftmost commit, not the rightmost. It also misses directionality arrows between commits.

See all responses

More from the list: "GIT Gold"

Curated by Pancake Agree

 In Level Up ... by Pushkar ...

6 Commands That Quietly Changed How I...

Feb 14

 Bhavyansh

10 Git Commands Every Developer Should Know...

★ · Oct 27, 2025



[View list](#)

More from Subodh Shetty and Stackademic



In Stackademic by Subodh Shetty

Git Rebase -Explained Like You're New to Git

When you work in Git, you'll often make a branch to build a feature or fix a bug. While you're working, other people keep adding new...



Aug 11, 2025



1.3K



21



In Stackademic by Gwang-Jin

Make Your Home Computer Reachable From Anywhere (No Public IP, No Router Touching)

Be set up in only ~10 minutes!



Feb 18



1.1K



10



In Stackademic by Ashish Singh

9 SSH Hardening Tricks That Stopped 50,000 Attack Attempts on My Server

How I went from dozens of daily breach attempts to virtually zero intrusions using these battle-tested configurations



Feb 16



654



13



 Subodh Shetty

RICE Model : A product feature prioritization technique for Engineering & Product managers

Every product team faces the same challenge: too many ideas, too little time. As an Engineering manager, I've often found myself looking...

★ Jun 19, 2025 🖱️ 18



See all from Subodh Shetty

See all from Stackademic

Recommended from Medium



In Mac O'Clock by Alex Gear & Tech Reviews

macOS is Good. These 9 Apps Make It Perfect.

M-Series Macs are monsters, but macOS itself still has many friction points. Here is how I “fix” it in 2026.



Feb 8



868



14



Will Lockett 

You Have No Idea How Far Behind Tesla Is

You have no idea how far behind you are

The leapfrog moment has just happened.

★ 6d ago 🖱️ 4.1K 💬 122



 Reza Rezvani

10 Claude Code Commands That Cut My Dev Time 60%: A Practical Guide

Custom slash commands, subagents, and automation workflows that transformed my team's productivity—with copy-paste templates you can use

★ Nov 20, 2025 🖱️ 1.91K 💬 35

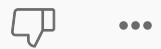


 Phil | Rentier Digital 

I Stopped Vibe Coding and Started “Prompt Contracts” — Claude Code Went From Gambling to Shipping

Last Tuesday at 2 AM, I deleted 2,400 lines of code that Claude Code had just generated for me.

 Feb 10  3.3K  90



 In AI Software Engineer by Joe Njenga

Claude Can Now Create Complex Diagrams — I Tested 21 Prompts

(Goodbye Canva!)

We have come along with Canva, but now Claude's new update does the job faster.

★ Mar 12 🖱️ 810 💬 8



● The Latency Gambler

Anthropic Says Engineers Won't Exist in a Year. It's Also Paying Them \$570K Today.

The most honest job posting in tech history might also be the most revealing thing about where the industry actually stands.

★ Mar 11 🖱️ 325 💬 12



See more recommendations